

Feed de Instagram con Jetpack Compose

Respuestas a las preguntas de reflexión

Estudiante: Ruben Esteban Mariaca Bautista | **Código:** U00185954
Ingeniería de Sistemas – Universidad Autónoma de Bucaramanga (UNAB)
Docente: Mg. Ing. Fabián Enrique Suárez Carvajal

Sección 00 – Paquetes

1. ¿Por qué es importante separar el código en paquetes desde el inicio del proyecto y no poner todo en el mismo lugar?

Porque el código queda organizado por responsabilidad (modelos, datos, pantallas y componentes). Así es más fácil encontrar cada archivo, entenderlo y modificarlo sin afectar otras partes, y el proyecto puede crecer sin volverse desordenado.

2. Si en el futuro se agrega una pantalla de perfil de usuario, ¿en qué paquete iría y cómo se llamaría?

En el paquete `ui.screens`, con el nombre `ProfileScreen.kt`.

Sección 01 – Data Class

1. ¿Qué pasaría si se usara una class normal en lugar de una data class? ¿Se podrían seguir usando `copy()` y `toString()` de forma automática?

No. Una class normal no genera `copy()`, y su `toString()` mostraría algo como `Post@1a2b3c` en lugar de los datos. Habría que escribir esos métodos a mano.

2. ¿Por qué todas las propiedades del Post usan `val` y no `var`? ¿Qué implicaría cambiar una a `var`?

Porque `val` hace el objeto inmutable y evita cambios accidentales; para modificarlo se crea una copia con `copy()`. Con `var` se podría cambiar directamente y Compose podría no detectar el cambio ni redibujar la pantalla.

Sección 02 – LazyColumn

1. ¿Para qué sirve el parámetro `key` en `items()`? ¿Qué podría salir mal si no se usa cuando el usuario da like a un post?

El `key` identifica cada ítem de forma única. Sin él, si la lista cambia, Compose podría confundir los ítems y el estado (por ejemplo, el like) podría aparecer en un post equivocado.

2. ¿Cuándo se usaría `item { }` en lugar de `items() { }`? Ejemplo concreto del feed.

`item { }` se usa para un solo elemento, como la fila de stories o el texto "Inicio del Feed".
`items()` se usa para recorrer una lista, como los posts.

Sección 03 – LazyRow

1. ¿Por qué se usa LazyRow para las stories y no un Row simple? ¿Cuándo sería aceptable un Row normal?

Porque LazyRow solo dibuja las stories visibles, mientras que Row dibuja todas aunque no se vean. Un Row normal es aceptable para pocos elementos fijos, como los íconos de acciones de un post.

2. La condición del anillo es `if (!story.hasSeen)`. ¿Qué significa el `!` y cómo se escribiría en español?

El `!` es el operador de negación. Se lee: "si la historia NO ha sido vista".

Sección 04 – DataSource

1. ¿Qué es un object en Kotlin y por qué es útil para un DataSource? ¿Qué pasaría con una class normal y dos instancias?

Un object es un singleton: existe una sola instancia en toda la app. Con una class normal se podrían crear dos DataSource con datos duplicados o inconsistentes y se gastaría más memoria.

2. En una app real, ¿de dónde vendrían estos datos? ¿Qué cambiaría en la arquitectura?

Vendrían de una API REST (por ejemplo con Retrofit) o de una base de datos local (Room). Se agregaría una capa de repositorio y un ViewModel entre los datos y la pantalla.

Sección 05 – Coil

1. ¿Por qué se usa AsyncImage de Coil en lugar de Image() de Compose? ¿Qué limitación tiene Image()?

Image() solo muestra imágenes locales (recursos o bitmaps) y no puede cargar desde una URL. AsyncImage descarga la imagen de internet en segundo plano sin bloquear la app.

2. ¿Por qué es importante que Coil guarde las imágenes en caché en un feed como Instagram?

Porque evita descargar la misma imagen varias veces al hacer scroll. El feed se ve más rápido y fluido y consume menos datos móviles.

Sección 06 – PostCard

1. Si el usuario da like y luego el post sale de pantalla, ¿qué pasa con el like al volver? ¿Por qué?

Se pierde y vuelve al valor original, porque remember se descarta cuando el post sale de la composición. Para conservarlo habría que usar rememberSaveable o guardar el estado en un ViewModel.

2. ¿Por qué PostHeader, PostActions y PostFooter son private? ¿Qué pasaría si se quita?

Porque solo se usan dentro de PostCard.kt. Si se quita private, cualquier archivo podría usarlas; no daría error, pero se pierde el encapsulamiento y el orden del código.

Reflexión final

1. ¿Por qué las stories van dentro de un item {} del LazyColumn y no antes? ¿Qué problema causaría ponerlas fuera?

Para que se desplacen junto con el feed, como en Instagram. Fuera del LazyColumn quedarían fijas arriba ocupando espacio o se generarían conflictos de scroll entre listas.

2. ¿Qué carpeta tocar para cambiar los datos del feed? ¿Y para cambiar cómo se ve un post?

Para los datos: `data (DataSource.kt)`. Para la apariencia de un post: `ui/components (PostCard.kt)`.

3. Si la app tuviera 1000 posts, ¿el LazyColumn podría manejarlo? ¿Por qué?

Sí, porque `LazyColumn` solo compone y dibuja los posts visibles y reutiliza los espacios al hacer scroll, así que no carga los 1000 a la vez.